



[WHS4기 컴퓨터구조1] 박성현 (4225)

📅 작성일	2026년 6월 16일
📄 과목명	컴퓨터구조1
⚙️ 과제	완료
≡ 이름	4기 39반 박성현

컴퓨터 구조 1 과제

1. 깃헙 코드스페이스로 리눅스 개별환경 만들기

2. 오버플로 예제를 언더플로로 바꿔서 해보기

- CHAR_MIN 의 값에서 -1

3. 비트 연산 프로그램 바꿔보기

- 특정 위치의 비트를 끄는 함수 구현
- + 사용자의 입력(특정 위치 -int값)을 받도록 수정 (선택)

4. C언어가 기계어가 되는 과정 직접 해보기

1. 깃헙 코드스페이스로 리눅스 개별환경 만들기

Github repository 생성

새로운 저장소

검색하려면 입력하세요

+

🔍

🔖

📄

📁

🌐

새 저장소를 생성하세요

저장소에는 프로젝트 파일과 버전 기록이 포함되어 있습니다. 다른 곳에 프로젝트가 있나요? [저장소를 가져오세요](#).
필수 입력 항목에는 별표(*)가 표시되어 있습니다.

1 일반적인

소유자 *

tjdus2670

저장소 이름 *

WHS_computerArchitecture1

WHS_computerArchitecture1을 사용할 수 있습니다.

훌륭한 저장소 이름은 짧고 기억하기 쉽습니다. 예를 들어, 다음과 같은 이름은 어떨까요? [verbose-octo-chainsaw?](#)

설명

ComputerArchotecture_assignment

0 / 350 자

2 구성

표시 여부를 선택하세요 *

이 저장소를 보고 커밋할 수 있는 사용자를 선택하세요.

사적인

README 파일을 추가하세요

README 파일은 자세한 설명으로 사용할 수 있습니다. README 파일에 대하여

끄다

.gitignore 파일을 추가하세요

.gitignore 파일은 Git에게 추적하지 않을 파일을 알려줍니다. 무시할 파일에 대한 자세한 내용은 다음을 참조하세요.

.gitignore 파일이 없습니다.

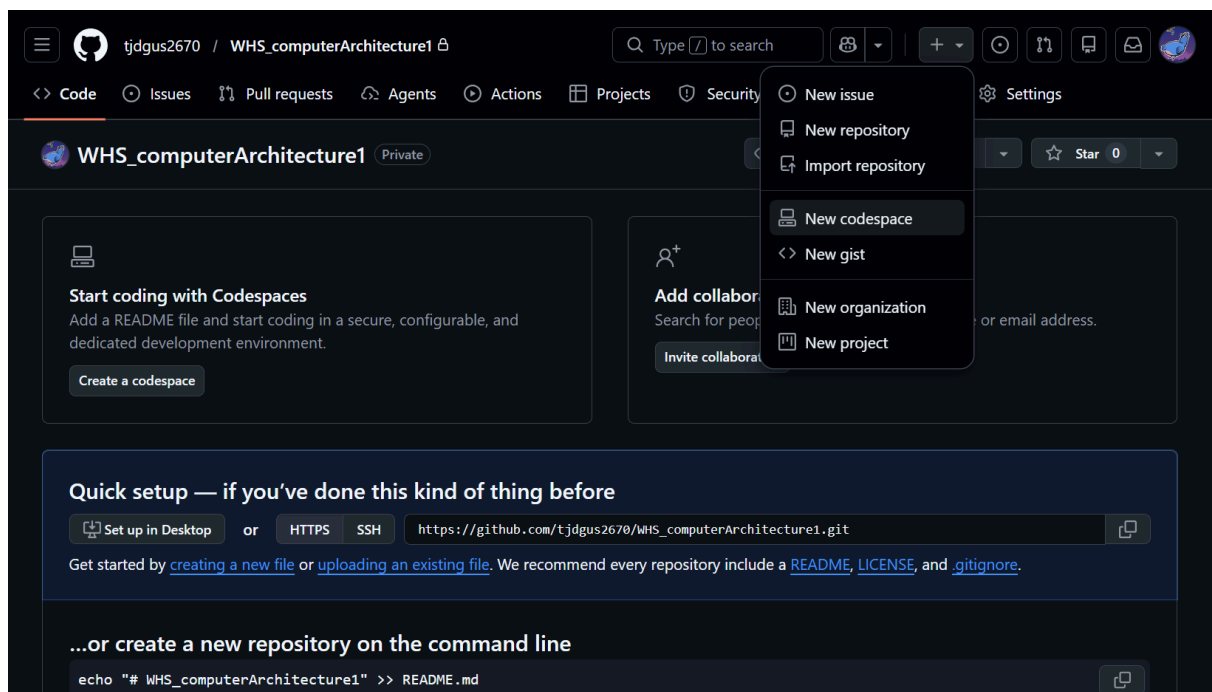
라이선스 추가

라이선스는 다른 사람들이 여러분의 코드를 어떻게 사용할 수 있는지 설명합니다. 라이선스에 대하여

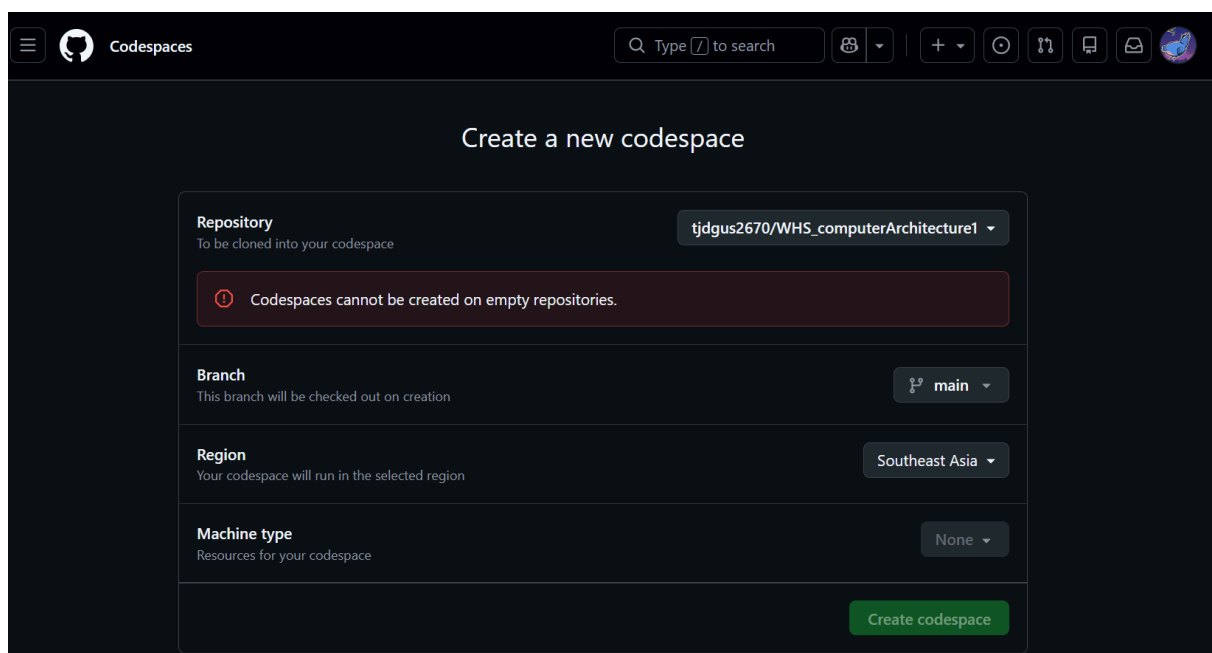
라이선스 없음

저장소 생성

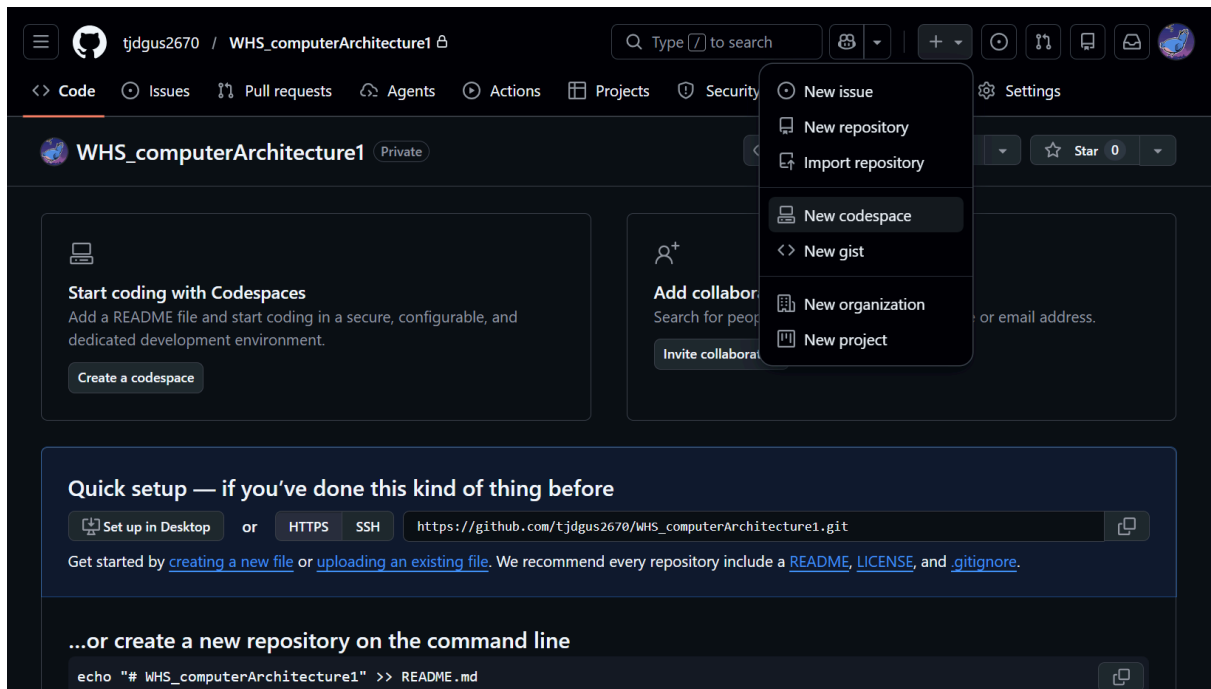
New codespace 클릭



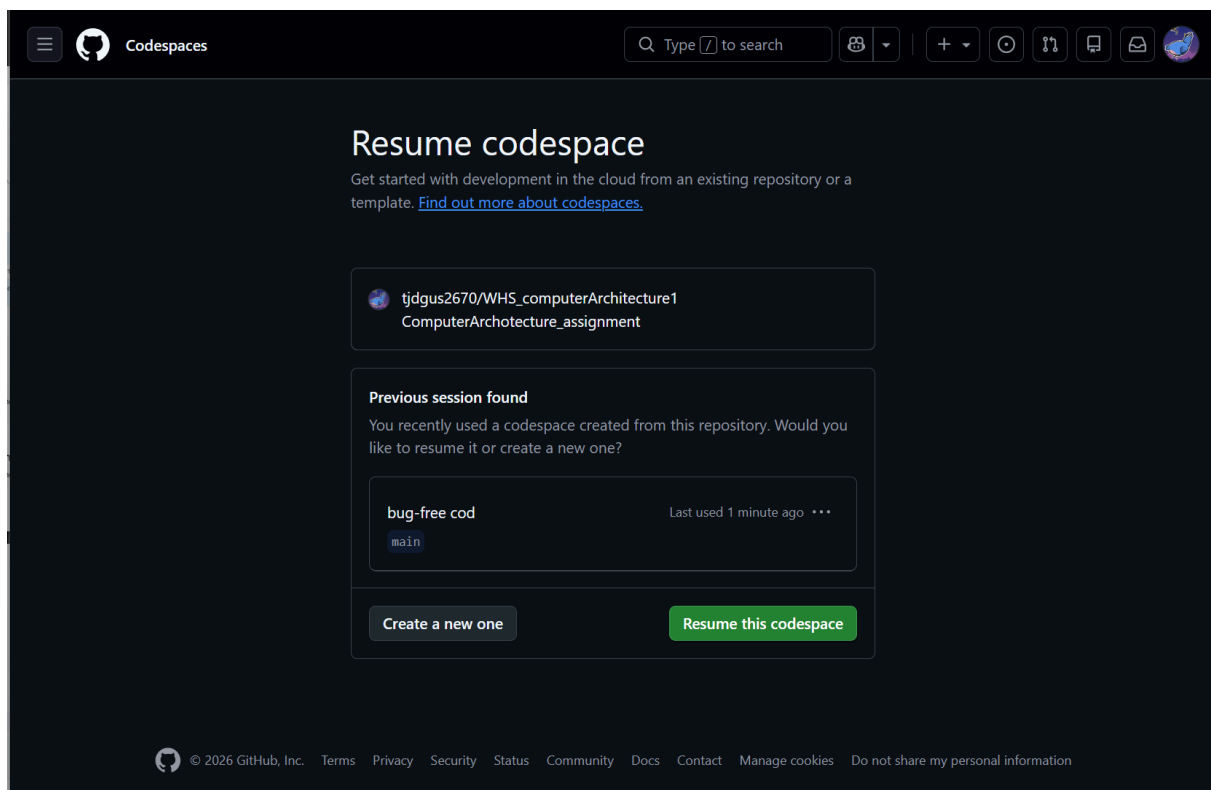
repository가 비어있어서 코드스페이스가 실행되지 않음



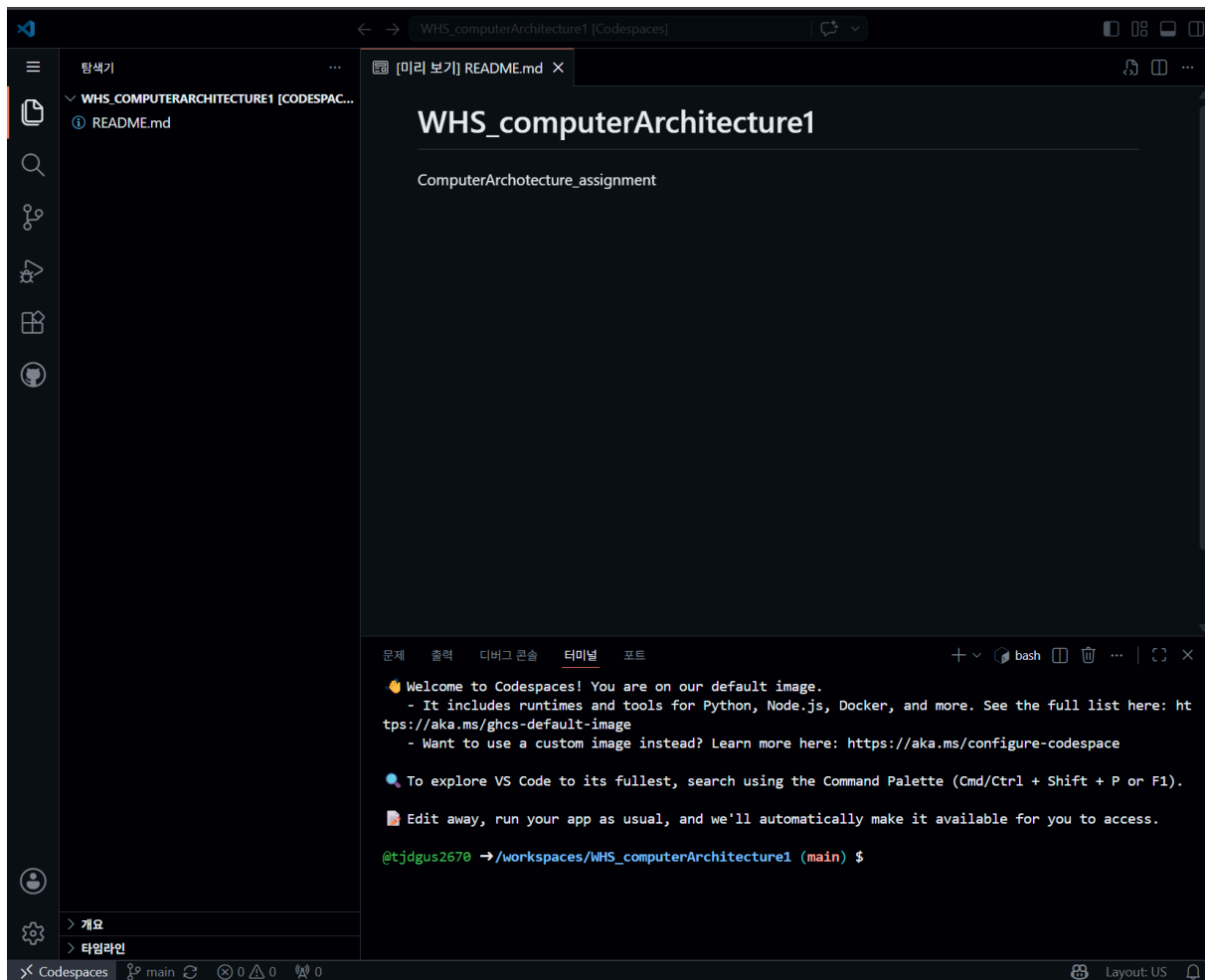
가운데에 Create a codespace 클릭



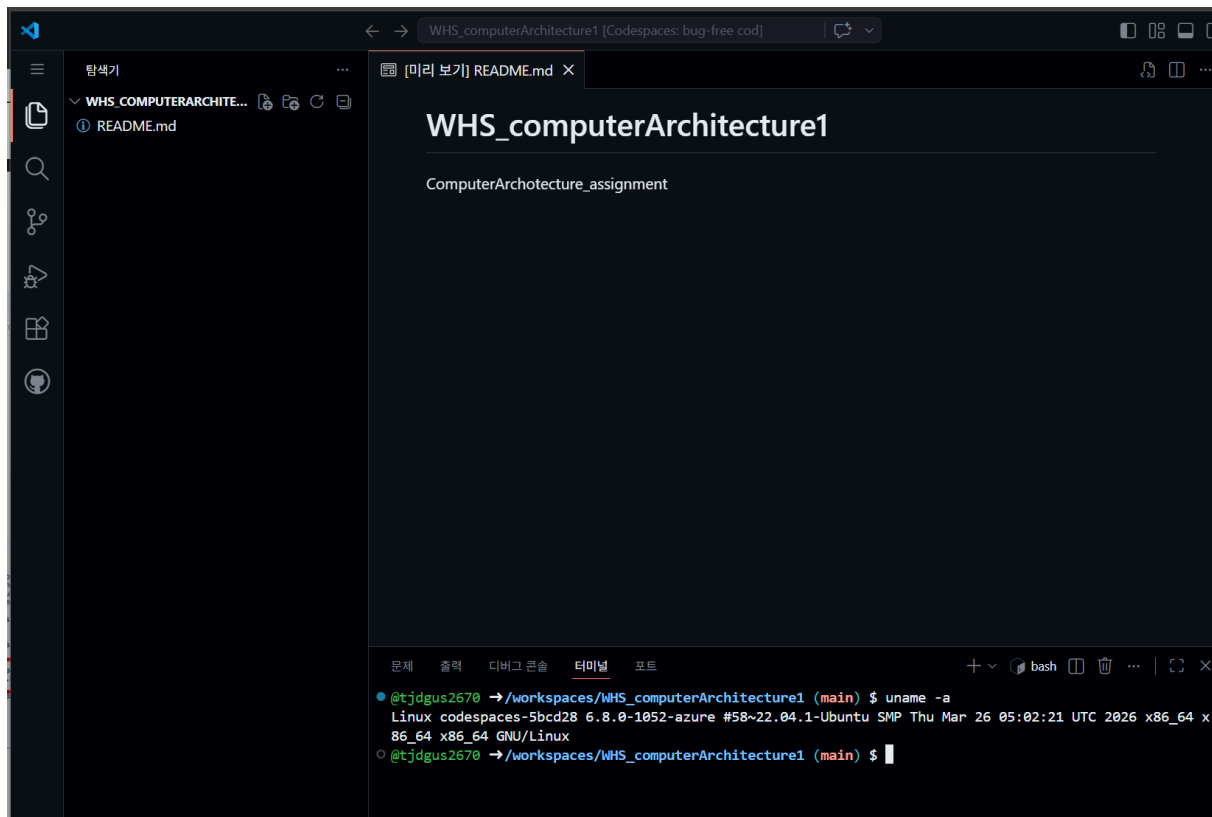
코드스페이스를 만들 수 있다



새로운 CodeSpace 생성



ubuntu 22.04 를 확인



2. 오버플로 예제를 언더플로로 바꿔서 해보기

- CHAR_MIN 의 값에서 -1

기존 오버플로우 예제 코드는 다음과 같다

```
#include <stdio.h>
#include <limits.h>

int main() {
    char value = CHAR_MAX;

    printf("Original value: %d\n", value);

    value = value + 1;

    printf("Value after adding 1:%d\n", value);
    return 0;
}
```

이중에서 CHAR_MAX는 변수의 최댓값을 출력해주는 기능이므로 CHAR_MIN을 사용 시 최솟값이 되고, +1을 -1로 교체해준다면 언더플로우가 발생하는 로직이 된다.

```
#include <stdio.h>
#include <limits.h>

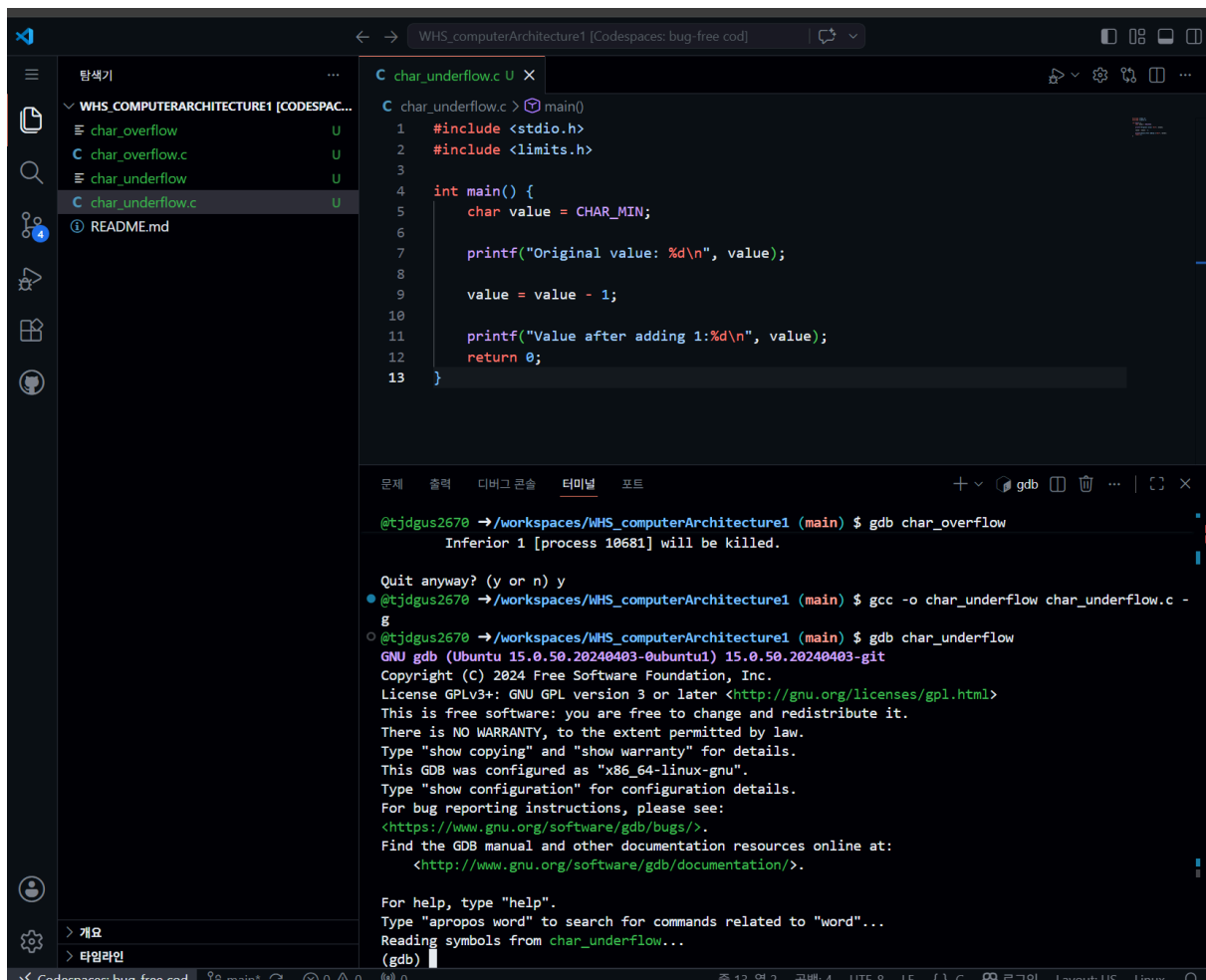
int main() {
    char value = CHAR_MIN;

    printf("Original value: %d\n", value);

    value = value - 1;

    printf("Value after adding 1:%d\n", value);
    return 0;
}
```

파일생성하여 코드 작성 후 gdb명령



The screenshot shows the Visual Studio Code editor with a workspace named 'WHS_computerArchitecture1'. The file explorer on the left shows a folder 'WHS_COMPUTERARCHITECTURE1 [CODESPACES]' containing files 'char_overflow.c', 'char_underflow.c', and 'README.md'. The editor window displays the code for 'char_underflow.c', which is identical to the code block above. The terminal at the bottom shows the following commands and output:

```
@tjdgus2670 → /workspaces/WHS_computerArchitecture1 (main) $ gdb char_overflow
Inferior 1 [process 10681] will be killed.

Quit anyway? (y or n) y
@tjdgus2670 → /workspaces/WHS_computerArchitecture1 (main) $ gcc -o char_underflow char_underflow.c -g
@tjdgus2670 → /workspaces/WHS_computerArchitecture1 (main) $ gdb char_underflow
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from char_underflow...
(gdb)
```

b main으로 분기를 나누고, r 명령으로 실행을 한다

```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ gdb char_underflow
(gdb) b main
Breakpoint 1 at 0x1155: file char_underflow.c, line 5.
(gdb) r
Starting program: /workspaces/WHS_computerArchitecture1/char_underflow

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at char_underflow.c:5
5      char value = CHAR_MIN;
```

printf 명령이 수행되고 오리지널 값이 -128이라고 출력된다.

여기서 -1 연산을 수행하였더니 1000_0000의 값이 111_1111이 되어 127로 언더플로우가 발생한다.

```
(gdb) n
7      printf("Original value: %d\n", value);
(gdb) n
Original value: -128
9      value = value - 1;
(gdb) p/t value
$1 = 100000000
(gdb) n
11     printf("Value after adding 1:%d\n", value);
(gdb) p/t value
$2 = 1111111
(gdb) p value
$3 = 127 '\177'
(gdb) █
```

3. 비트 연산 프로그램 바꿔보기

- 특정 위치의 비트를 끄는 함수 구현
- + 사용자의 입력(특정 위치 -int값)을 받도록 수정 (선택)

예제 코드에서 특정 위치의 비트를 끄는 함수를 만들어야 한다

아래의 코드를 추가한다. 비트를 변환할 위치를 선정해주고, 3번째면 0000_1000 같은 형식으로 1의 위치를 결정 후, 이를 not 연산으로 비트반전 후, and 연산을 하면 원하는 위치의 비트를 끌 수 있다.


```

unsigned char clear_bit(unsigned char value, int position) {
    return value & ~(1 << position);
}

```

전체 코드는 다음과 같다. (사용자에게 scanf를 통해 값을 받도록 수정)

```

#include <stdio.h>

int is_bit_set(unsigned char value, int position) {
    return (value & (1 << position)) != 0;
}

unsigned char set_bit(unsigned char value, int position) {
    return value | (1 << position);
}

unsigned char clear_bit(unsigned char value, int position) {
    return value & ~(1 << position);
}

int main() {
    unsigned char value = 0b00001000;
    unsigned char value1 = 0b00001000;
    int n = 0;

    printf("set the bit:");
    scanf("%d", &n);

    if (is_bit_set(value, 3)){
        printf("3rd bit is set!\n");
    }else {
        printf("3rd bit is not set!\n");
    }

    value = set_bit(value, 2);
    printf("Value after setting 2nd bit: %d\n", value);

    value = clear_bit(value1, n);

    printf("Value after clearing %d bit: ", n);
}

```

```

    for (int i = 7; i >= 0; i--) {
        printf("%d", (value >> i) & 1);
    }
    printf("\n");

    return 0;
}

```

The screenshot shows a Visual Studio Code editor with a C program file named `260613_ComputerA1.c`. The code defines functions `is_bit_set`, `set_bit`, and `clear_bit`, and a `main` function that tests these operations on a value of 12. The terminal output shows the program's execution, including the results of bit setting and clearing operations.

```

22
23
24     if (is_bit_set(value, 3)){
25         printf("3rd bit is set!\n");
26     }else {
27         printf("3rd bit is not set!\n");
28     }
29
30     value = set_bit(value, 2);
31     printf("Value after setting 2nd bit: %d\n", value);
32
33     value = clear_bit(value, 1);
34
35     printf("Value after clearing %d bit: ", n);
36     for (int i = 7; i >= 0; i--) {
37         printf("%d", (value >> i) & 1);
38     }
39     printf("\n");
40
41     return 0;
42 }

```

Terminal Output:

```

2 -wcast-align -Wconversion -Wsign-conversion -Wnull-dereference -g3 -O0 .\build\Debug\260613_ComputerA1.o -o .\build\Debug\outDebug.exe
터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.

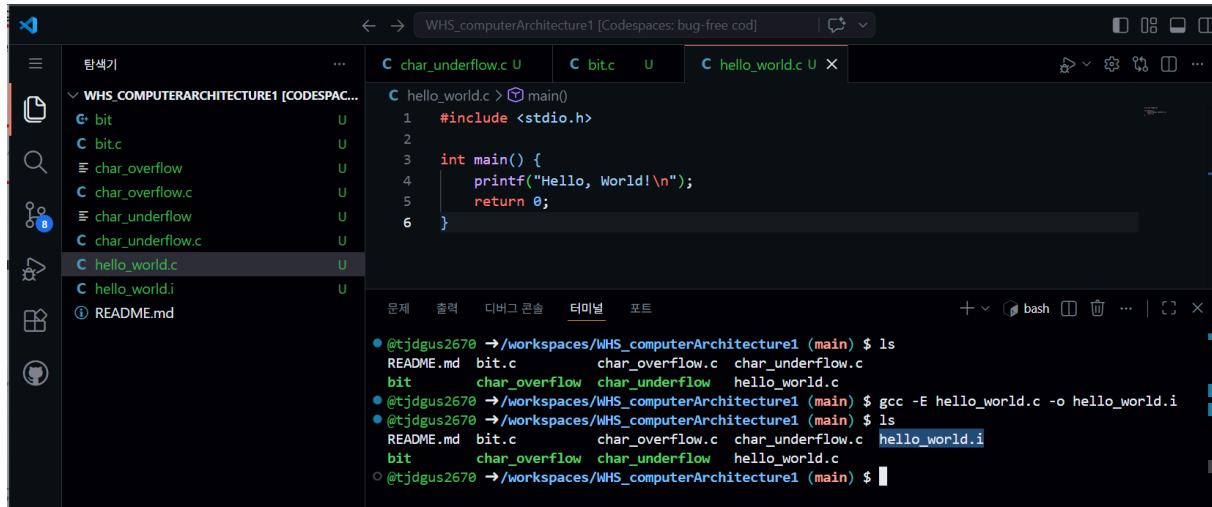
작업 실행 중: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

set the bit:3
3rd bit is set!
Value after setting 2nd bit: 12
Value after clearing 3 bit: 00000000
터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.

```

4. C언어가 기계어가 되는 과정 직접 해보기

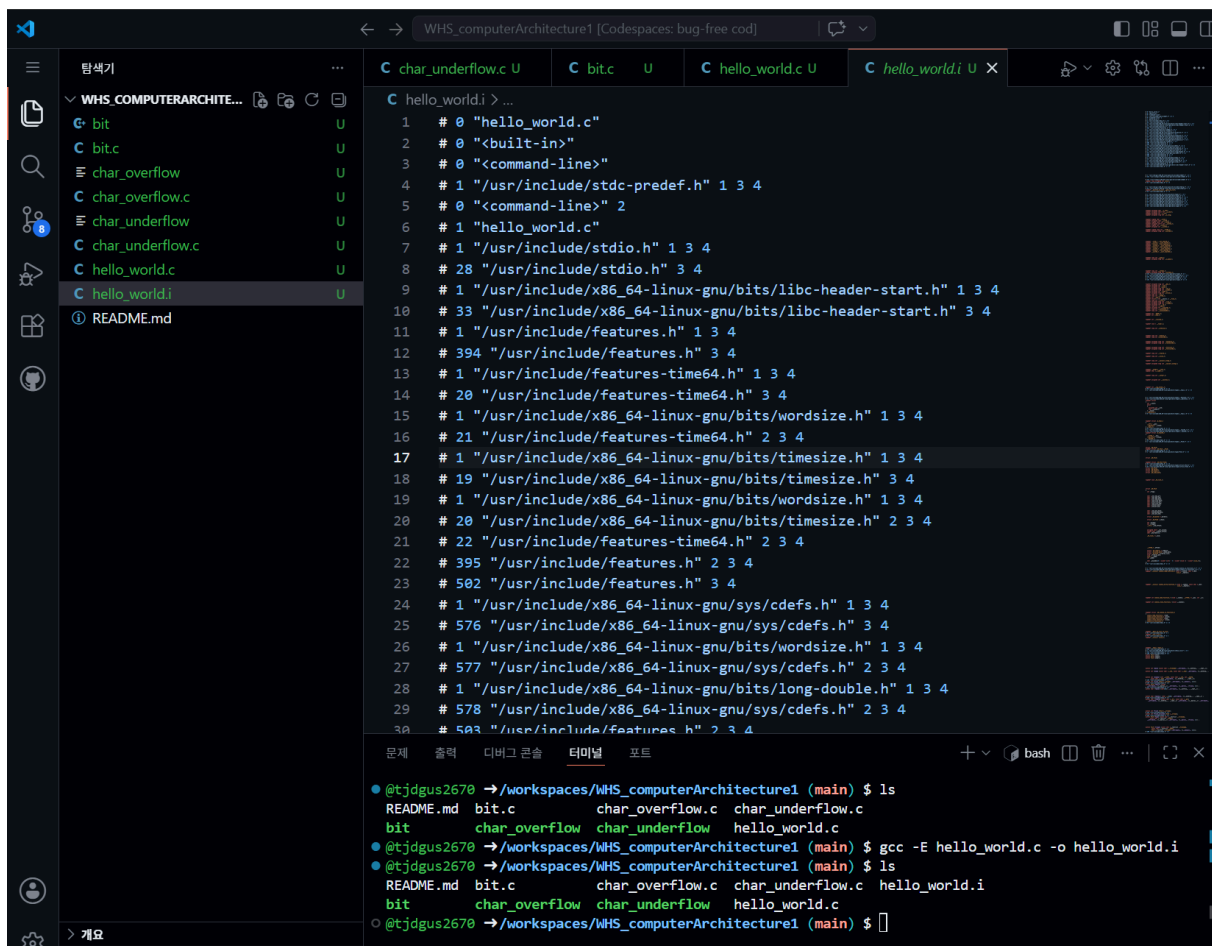
hello_world 작성 후, gcc 컴파일러로 i 파일을 생성



The screenshot shows the Visual Studio Code interface with a workspace named 'WHS_computerArchitecture1'. The file explorer on the left lists several files: bit, bit.c, char_overflow, char_overflow.c, char_underflow, char_underflow.c, hello_world.c, hello_world.i, and README.md. The main editor displays the source code of 'hello_world.c', which includes a standard C program with a main function that prints 'Hello, World!'. Below the editor, the integrated terminal shows the execution of the following commands:

```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ls
README.md bit.c char_overflow.c char_underflow.c
bit char_overflow char_underflow hello_world.c
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ gcc -E hello_world.c -o hello_world.i
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ls
README.md bit.c char_overflow.c char_underflow.c hello_world.i
bit char_overflow char_underflow hello_world.c
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```

hello_world.i 파일 확인 가능



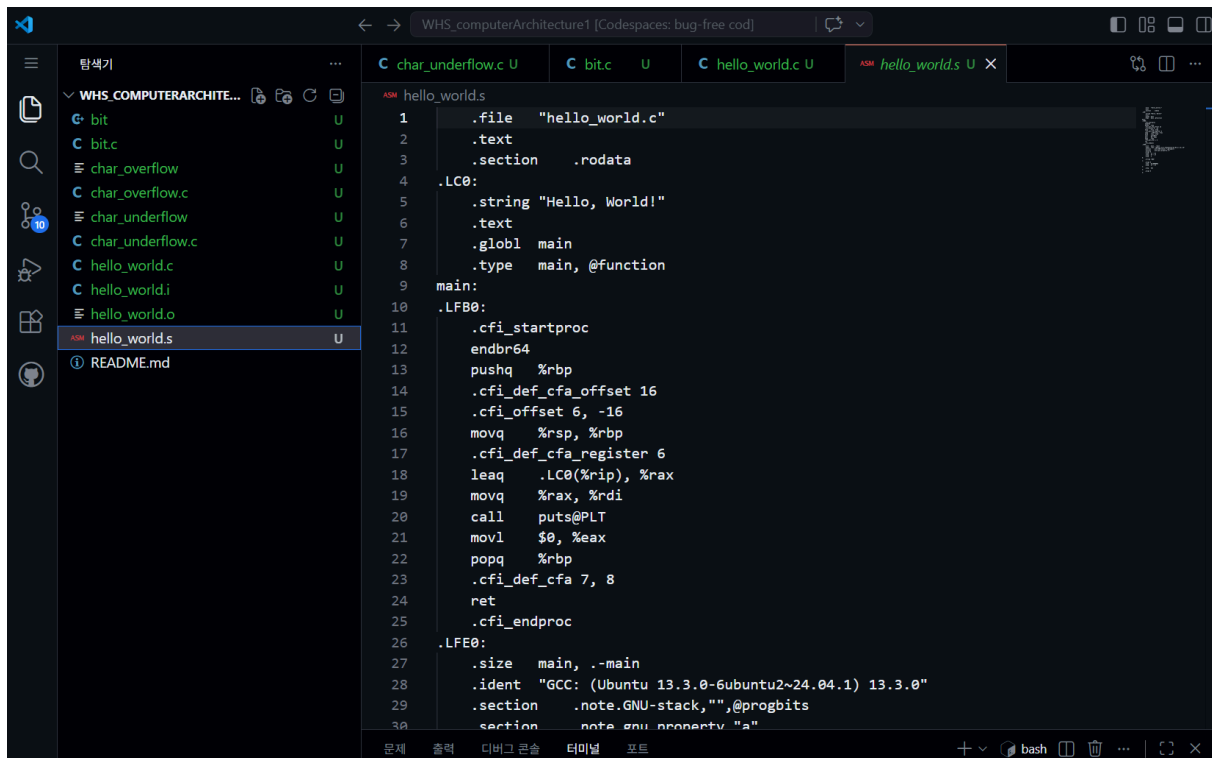
The screenshot shows the Visual Studio Code interface with the same workspace. The file explorer now includes 'hello_world.i'. The main editor displays the contents of 'hello_world.i', which is the preprocessor output of the original C file. It shows the inclusion of system headers like 'stdio.h' and 'x86_64-linux-gnu/bits/libc-header-start.h', along with the expanded preprocessor directives. The integrated terminal shows the following commands:

```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ls
README.md bit.c char_overflow.c char_underflow.c
bit char_overflow char_underflow hello_world.c
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ gcc -E hello_world.c -o hello_world.i
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ls
README.md bit.c char_overflow.c char_underflow.c hello_world.i
bit char_overflow char_underflow hello_world.c
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```

hello_world.s 파일 생성

```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ gcc -S hello_world.i -o hello_world.s
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ls
README.md bit.c char_overflow.c char_underflow.c hello_world.i
bit char_overflow char_underflow hello_world.c hello_world.s
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```

파일 확인 가능

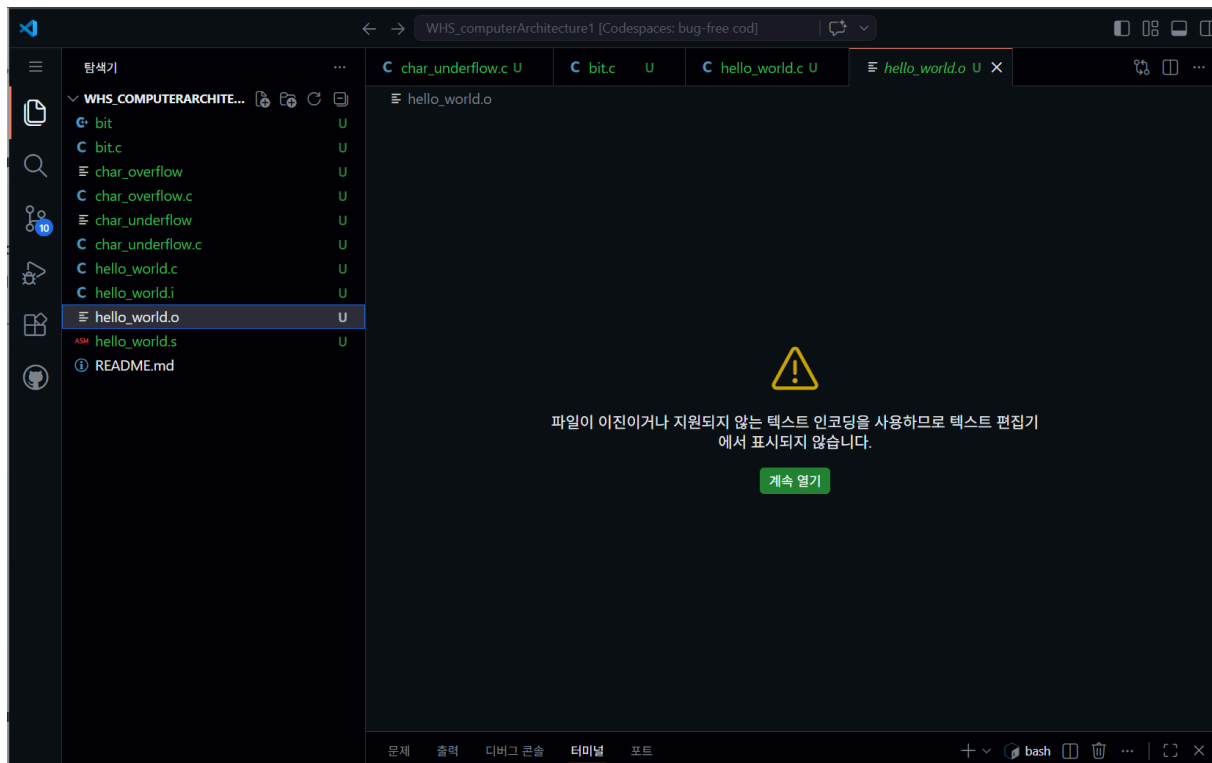


```
WHS_computerArchitecture1 [Codespaces: bug-free cod]
C char_underflow.c U C bit.c U C hello_world.c U ASM hello_world.s U
WHS_COMPUTERARCHITE...
bit U
bit.c U
char_overflow U
char_overflow.c U
char_underflow U
char_underflow.c U
hello_world.c U
hello_world.i U
hello_world.o U
ASM hello_world.s U
README.md
hello_world.s
1 .file "hello_world.c"
2 .text
3 .section .rodata
4 .LC0:
5 .string "Hello, World!"
6 .text
7 .globl main
8 .type main, @function
9 main:
10 .LFB0:
11 .cfi_startproc
12 endbr64
13 pushq %rbp
14 .cfi_def_cfa_offset 16
15 .cfi_offset 6, -16
16 movq %rsp, %rbp
17 .cfi_def_cfa_register 6
18 leaq .LC0(%rip), %rax
19 movq %rax, %rdi
20 call puts@PLT
21 movl $0, %eax
22 popq %rbp
23 .cfi_def_cfa 7, 8
24 ret
25 .cfi_endproc
26 .LFE0:
27 .size main, .-main
28 .ident "GCC: (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0"
29 .section .note.GNU-stack,"",@progbits
30 .section .note.gnu.property,"a"
```

hello_world.o 파일 생성

```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ gcc -c hello_world.s -o hello_world.o
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ls
README.md bit.c char_overflow.c char_underflow.c hello_world.i hello_world.s
bit char_overflow char_underflow hello_world.c hello_world.o
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```

파일을 열 수 없음



오브젝트 파일은 소스코드가 컴파일된 후의 중간 결과물로, 링커에 의해 실행 가능한 바이너리나 라이브러리로 만들어지기 전의 형태이다. readelf 도구로 읽어볼 것임

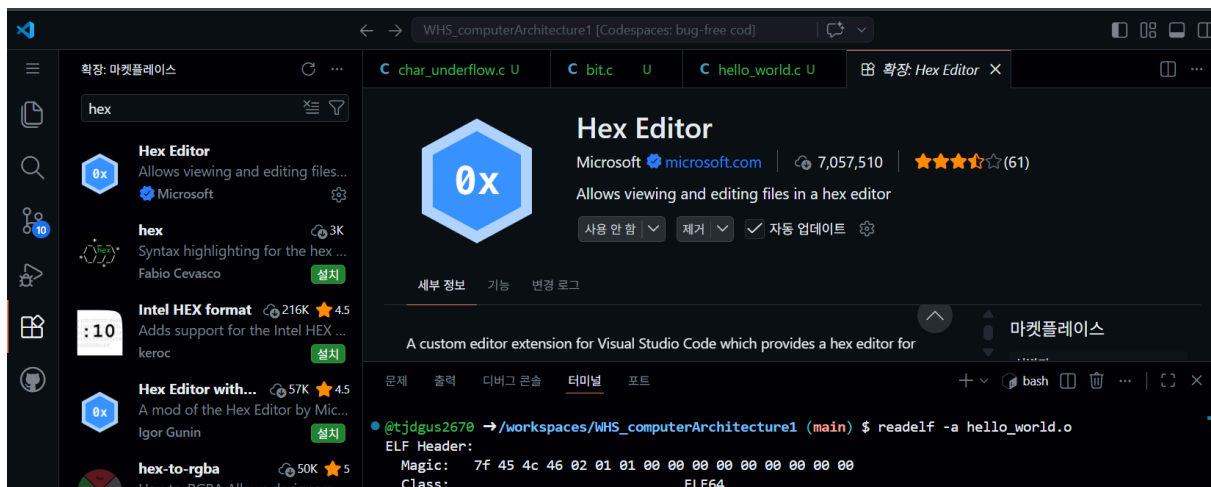
```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ file *
README.md:      ASCII text
bit:            ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=6f616270de07d19194433896a0ddbe1f51794431, for GNU/Linux 3.2.0, not stripped
bit.c:          C source, ASCII text
char_overflow:  ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=1718507154a77ae7920207e9949fe95947098e51, for GNU/Linux 3.2.0, with debug_info, not stripped
char_overflow.c: C source, ASCII text
char_underflow: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=cb00aa959044e4a1844a5a3bfac0d401f966a6c9, for GNU/Linux 3.2.0, with debug_info, not stripped
char_underflow.c: C source, ASCII text
hello_world.c:  C source, ASCII text
hello_world.i:  C source, ASCII text
hello_world.o:  ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
hello_world.s:  assembler source, ASCII text
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```

readelf 명령을 통해 내부 확인

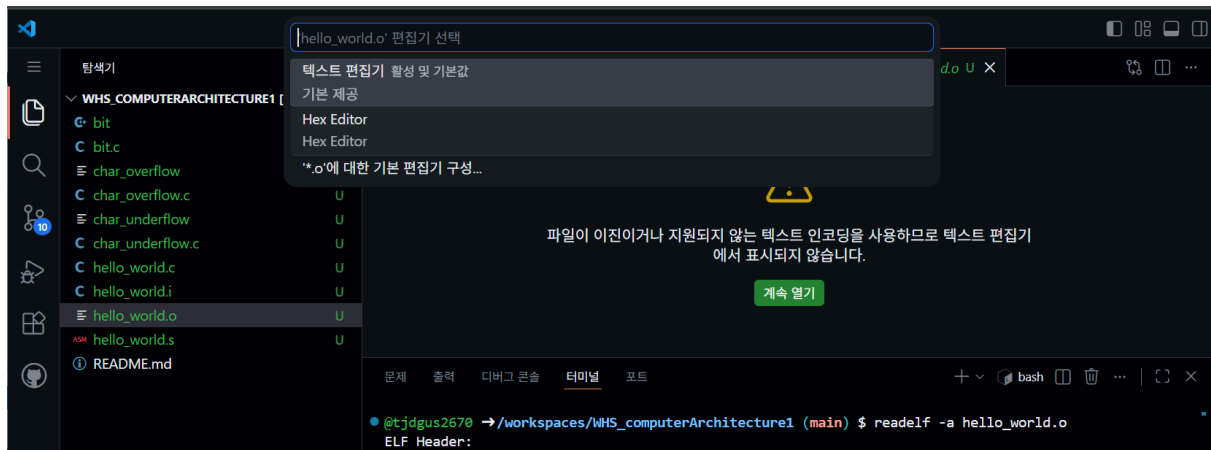
```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ readelf -a hello_world.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  REL (Relocatable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x0
  Start of program headers:              0 (bytes into file)
  Start of section headers:              616 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               0 (bytes)
  Number of program headers:              0
  Size of section headers:               64 (bytes)
  Number of section headers:              14
  Section header string table index:     13

Section Headers:
 [Nr] Name              Type              Address             Offset
     Size              EntSize          Flags  Link  Info  Align
 [ 0] 0000000000000000     NULL             0000000000000000   00000000
     0000000000000000  0000000000000000           0     0     0
```

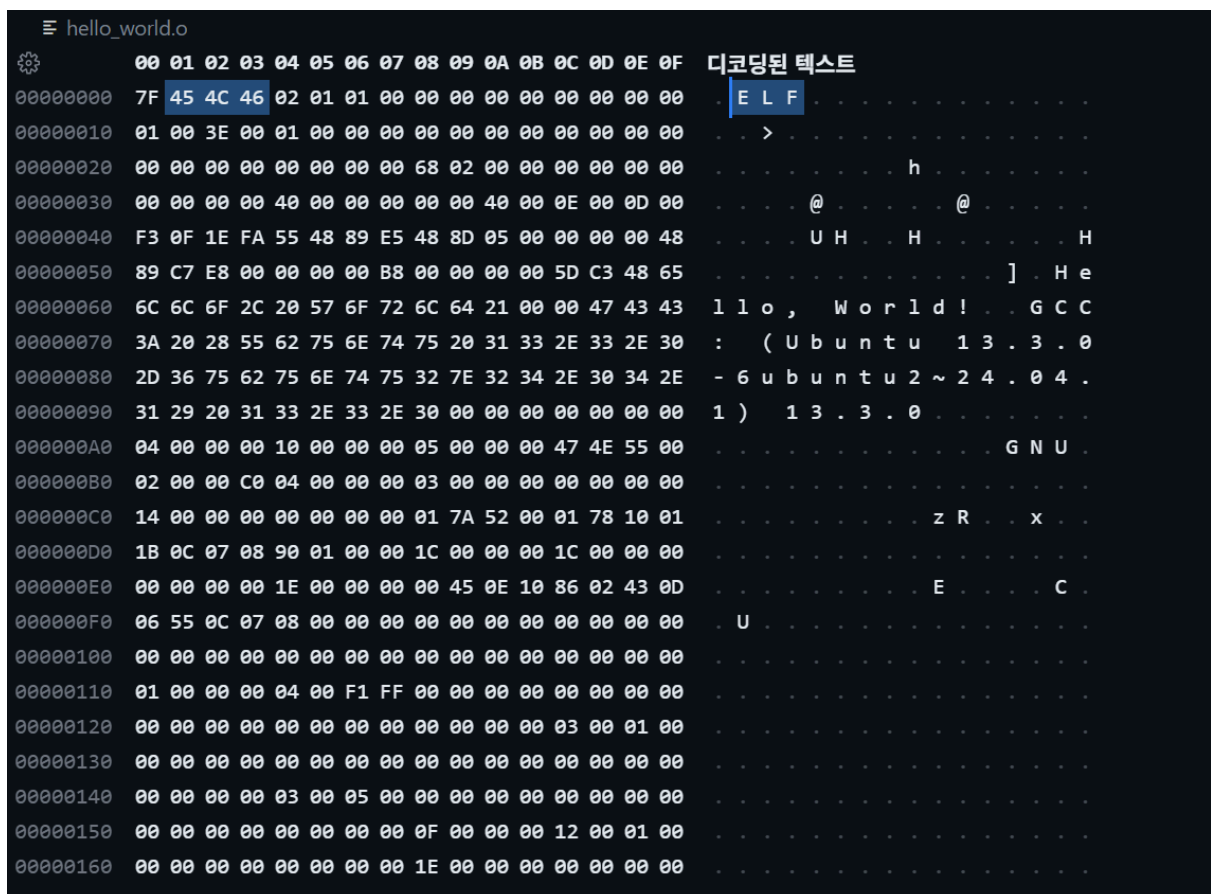
hex editor 설치



hex editor로 실행



hex editor를 통해 파일을 확인 가능하다



objdump를 통해 확인

```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ objdump -d hello_world.o

hello_world.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa      endbr64
 4:  55               push  %rbp
 5:  48 89 e5         mov   %rsp,%rbp
 8:  48 8d 05 00 00 00 lea   0x0(%rip),%rax      # f <main+0xf>
 f:  48 89 c7         mov   %rax,%rdi
12:  e8 00 00 00 00   call  17 <main+0x17>
17:  b8 00 00 00 00   mov   $0x0,%eax
1c:  5d               pop   %rbp
1d:  c3               ret

@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```

hello_world.o 파일로 hello_world 파일을 완성

```
문제  출력  디버그 콘솔  터미널  포트
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ gcc hello_world.o -o hello_world
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ls
README.md  bit.c      char_overflow.c  char_underflow.c  hello_world.c  hello_world.o
bit         char_overflow  char_underflow  hello_world        hello_world.i  hello_world.s
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ ./hello_world
Hello, World!
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```

실행파일과 오브젝트 파일의 차이를 확인 가능

```
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $ file *
README.md:      ASCII text
bit:            ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interp
reter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=6f616270de07d19194433896a0ddb1f51794431, for GNU/Li
nux 3.2.0, not stripped
bit.c:          C source, ASCII text
char_overflow:  ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interp
reter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=1718507154a77ae7920207e9949fe95947098e51, for GNU/Li
nux 3.2.0, with debug_info, not stripped
char_overflow.c: C source, ASCII text
char_underflow: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interp
reter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=cb00aa959044e4a1844a5a3bfac0d401f966a6c9, for GNU/Li
nux 3.2.0, with debug_info, not stripped
char_underflow.c: C source, ASCII text
hello_world:    ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interp
reter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f1ea79f542a78567330db6469de48bff0bf621b9, for GNU/Li
nux 3.2.0, not stripped
hello_world.c:  C source, ASCII text
hello_world.i:  C source, ASCII text
hello_world.o:  ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
hello_world.s:  assembler source, ASCII text
@tjdgus2670 →/workspaces/WHS_computerArchitecture1 (main) $
```


파일 크기도 전부 다름

```
● @tjdgus2670 → /workspaces/WHS_computerArchitecture1 (main) $ ls -alh
total 136K
drwxrwxrwx+ 3 codespace root      4.0K Jun 16 02:46 .
drwxr-xrwx+ 5 codespace root      4.0K Jun 16 00:38 ..
drwxrwxrwx+ 8 codespace root      4.0K Jun 16 00:39 .git
-rw-rw-rw-  1 codespace root        60 Jun 16 00:38 README.md
-rwxrwxrwx  1 codespace codespace 16K Jun 16 01:54 bit
-rw-rw-rw-  1 codespace codespace 799 Jun 16 01:54 bit.c
-rwxrwxrwx  1 codespace codespace 17K Jun 16 00:52 char_overflow
-rw-rw-rw-  1 codespace codespace 212 Jun 16 00:51 char_overflow.c
-rwxrwxrwx  1 codespace codespace 17K Jun 16 00:59 char_underflow
-rw-rw-rw-  1 codespace codespace 212 Jun 16 00:58 char_underflow.c
-rwxrwxrwx  1 codespace codespace 16K Jun 16 02:46 hello_world
-rw-rw-rw-  1 codespace codespace  79 Jun 16 01:59 hello_world.c
-rw-rw-rw-  1 codespace codespace 21K Jun 16 02:06 hello_world.i
-rw-rw-rw-  1 codespace codespace 1.5K Jun 16 02:42 hello_world.o
-rw-rw-rw-  1 codespace codespace 674 Jun 16 02:11 hello_world.s
○ @tjdgus2670 → /workspaces/WHS_computerArchitecture1 (main) $
```

objdump결과를 통해 확인

```
● @tjdgus2670 → /workspaces/WHS_computerArchitecture1 (main) $ objdump -d hello_world

hello_world:      file format elf64-x86-64

Disassembly of section .init:

0000000000001000 <.init>:
   1000:    f3 0f 1e fa                endbr64
   1004:    48 83 ec 08                sub    $0x8,%rsp
   1008:    48 8b 05 d9 2f 00 00      mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_start__@Base>
   100f:    48 85 c0                  test   %rax,%rax
   1012:    74 02                     je     1016 <.init+0x16>
   1014:    ff d0                     call   *%rax
   1016:    48 83 c4 08                add    $0x8,%rsp
   101a:    c3                       ret

Disassembly of section .plt:
```

objdump -d 의 결과에서 gdb 디버거의 결과와 같음을 확인 가능

```
0000000000001149 <main>:
1149:    f3 0f 1e fa    endbr64
114d:    55             push    %rbp
114e:    48 89 e5       mov     %rsp,%rbp
1151:    48 8d 05 ac 0e 00 00    lea     0xeac(%rip),%rax    # 2004 <_IO_stdin_used+0x4>
1158:    48 89 c7       mov     %rax,%rdi
115b:    e8 f0 fe ff ff    call    1050 <puts@plt>
1160:    b8 00 00 00 00    mov     $0x0,%eax
1165:    5d             pop     %rbp
1166:    c3             ret

Disassembly of section .fini:
```

```
(gdb) disassemble main
Dump of assembler code for function main:
0x0000000000001149 <+0>:    endbr64
0x000000000000114d <+4>:    push    %rbp
0x000000000000114e <+5>:    mov     %rsp,%rbp
0x0000000000001151 <+8>:    lea     0xeac(%rip),%rax    # 0x2004
0x0000000000001158 <+15>:   mov     %rax,%rdi
0x000000000000115b <+18>:   call    0x1050 <puts@plt>
0x0000000000001160 <+23>:   mov     $0x0,%eax
0x0000000000001165 <+28>:   pop     %rbp
0x0000000000001166 <+29>:   ret
End of assembler dump.
(gdb)
```

gdb 디버거를 통해 확인 시 실제 메모리 주소가 잡히는것을 확인 가능

```
@tjdgus2670 → /workspaces/WHS_computerArchitecture1 (main) $ gdb hello_world
Breakpoint 1 at 0x1151
(gdb) r
Starting program: /workspaces/WHS_computerArchitecture1/hello_world
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000555555551151 in main ()
(gdb) disassemble main
Dump of assembler code for function main:
0x0000555555551149 <+0>:    endbr64
0x000055555555114d <+4>:    push    %rbp
0x000055555555114e <+5>:    mov     %rsp,%rbp
=> 0x0000555555551151 <+8>:    lea     0xeac(%rip),%rax    # 0x555555556004
0x0000555555551158 <+15>:   mov     %rax,%rdi
0x000055555555115b <+18>:   call    0x555555555050 <puts@plt>
0x0000555555551160 <+23>:   mov     $0x0,%eax
0x0000555555551165 <+28>:   pop     %rbp
0x0000555555551166 <+29>:   ret
End of assembler dump.
(gdb)
```